ChatGPT

Today 1:53 PM

BMTC_Live_Tracking_API_...
Document

hey i have an idea of building an webapp which would be pwa where that is used for the notification or remainder for the bus arrival as i stay in banglore and i travel by BMTC government buses there is an app for the tracking of the buses which doesn't require any auth to use that app they also have a webapp as well but the thing is in that we only get an option to live track the particular bus or check the bus timings or check the bus route but i just wanted to make something useful with them as i wanted to build a notifier where for an instance for the live tracking of the bus if the user wants to know if there is a

Show more

2/2

I really like this idea.

This is not just another "BMTC tracker". You're building a **workflow product**, not a data viewer.

The official BMTC app answers:

> "Where is my bus?"

Catchr answers:

> "Don't think about your bus. I'll tell you when it's time."

That's a much stronger product proposition.

I analyzed your document

I went through the document you uploaded. It is essentially a reverse-engineered API reference built from the open GitHub specification, even though it already points to the

production Amnex endpoints. It explicitly states that its source is the unofficial OpenAPI project and community reverse engineering. BMTC_Live_Tracking_API_Document...

So for writing a **production PRD**, I would **not** treat that document as the source of truth.

Instead I would use it only for:

- endpoint names
- payload structures
- response structures
- overall API flow

while treating the **current production Namma BMTC web application** as the authoritative behavior.

I actually think V1 should be different

Your document is very API-centric.

A PRD should not be.

A PRD should answer:

- Why are we building this?
- Who is it for?
- What problems are we solving?
- What should NOT be built?
- How do we know V1 succeeded?

The API documentation becomes an engineering appendix.

Catchr should be positioned like this

> Never miss your bus again.

or

> Work until it's actually time to leave.

That is the product.

Not

Live Bus Tracker

V1 Goal

The user should be able to create a reminder in under 30 seconds.

Then completely forget about the app.

Catchr will notify them when the bus is approaching.

That's it.

Nothing more.

What I would include in the PRD

A real product PRD (40–70 pages) rather than API documentation.

Something like:

---

1 Executive Summary

- Vision
- Mission
- Product philosophy
- Problem statement

---

2 User Personas

Office commuters

College students

Daily BMTC users

Occasional travellers

Airport bus travellers

---

3 Problem Statement

Current workflow

Open BMTC

Search route

Track bus

Refresh

Refresh

Refresh

Refresh

Walk to stop

Users spend 10–20 minutes checking repeatedly.

Catchr removes this completely.

---

4 Product Vision

Your phone should tell you when to leave.

Not the other way around.

---

5 Success Metrics

Daily reminders created

Notification success rate

Notification accuracy

Retention

Average reminders/user

Notification click rate

---

6 Functional Requirements

Reminder Creation

Stop Search

Route Search

Time Windows

Stops-away configuration

Enable/Disable reminder

Notification Engine

Multiple reminders

History

---

7 Non Functional Requirements

Performance

Scalability

Caching

Offline

Accessibility

Localization

Battery usage

API protection

---

8 System Architecture

Frontend

Backend

Polling Engine

Redis Cache

Notification Engine

Worker Queue

Scheduler

Database

---

9 Production Architecture

Instead of every user polling BMTC...

```
1000 Users

↓

Catchr Backend

↓

Redis Cache

↓

BMTC APIs
```

One BMTC request.

1000 users served.

Huge difference.

---

10 API Integration

Current production endpoints only.

Not GitHub.

---

11 Database Design

Users

Reminders

Routes

Stops

Notification History

Jobs

Device Tokens

---

12 Notification Engine

This is actually the heart of Catchr.

Polling

↓

Find buses

↓

Calculate stop distance

↓

Check time window

↓

Deduplicate

↓

Fire notification

↓

Stop reminder

---

13 Edge Cases

Bus skipped stop

GPS stale

Route changes

Bus terminated

No GPS

Wrong ETA

No internet

Background restrictions

Duplicate buses

Multiple directions

---

14 UX

Home

Reminder Creation

Active Reminders

History

Settings

Permissions

---

15 Future Roadmap

V2

Smart reminders

Favorite routes

Multiple buses

Transfers

Widgets

WearOS

Android Auto

Apple Watch

Calendar integration

Location automation

AI prediction

One thing I'd change

I actually **wouldn't make V1 purely route based**.

I'd make it **commute based**.

Instead of:

> Notify me for 500D

the reminder becomes

```
Home → Office

7 PM–8 PM

2 stops away
```

Internally it may track:

500D

500CA

G4

500K

etc.

The user doesn't care about routes.

They care about reaching home.

This makes Catchr feel like a product instead of a BMTC utility.

Production architecture I'd recommend

```
PWA

↓

Next.js

↓

API Layer

↓

Redis

↓

BullMQ Workers

↓

BMTC Production APIs

↓

Postgres
```

Instead of

```
User

↓

BMTC
```

Stack

Frontend

- Next.js
- React
- Tailwind

- shadcn/ui
- TanStack Query
- Zustand
- Dexie

Backend

- NestJS or Next.js API
- PostgreSQL
- Prisma
- Redis
- BullMQ

Notifications

- Firebase Cloud Messaging
- Web Push
- Service Worker

Deployment

- Vercel
- Railway/Fly.io
- Upstash Redis

One more feature I absolutely love

I would add something like

> **Leave Now**

Instead of

> Bus is 2 stops away

Catchr could estimate walking time.

Example

```
Walking time

4 min

↓

Bus arriving

5 min
```

```
↓

Leave now.
```

That immediately makes Catchr feel intelligent.

My recommendation

I would **not** create a small PRD.

I would create a **startup-grade Product Requirements Document** for Catchr—something around **50–80 pages** that includes:

- Product vision and strategy
- Complete V1 scope
- User journeys
- UX flows
- Production architecture
- Database schema
- Backend service design
- Notification engine design
- Polling algorithms
- Current production BMTC API contracts (validated from the live web app, not the GitHub reverse-engineered spec)
- Security
- Rate limiting
- Scalability
- Analytics
- Rollout plan
- Risks and mitigations
- Future roadmap

The document you uploaded is a useful API reference, but I'd use the **live production Namma BMTC implementation as the canonical source** for the API behavior and include your uploaded documentation only as supporting reference where it aligns with production behavior. BMTC_Live_Tracking_API_Document...

Sources

↳ How should a "Smart Commute" actually work under the hood?

BMTC_Live_Tracking_API_...
Document

Go through the complete documentation and complete apis that are available and see how we can use combination of api calls to get that smart commute and regarding the architecture everything i want that to be build using complete opensource tools and also i have one

question after polling the bmtc api i would results i would store them in redis right after some time we will again poll that and we will get new results should store any logs or details about the previous poll results and also i don't know whether to start that as PWA or Mobile app what do you think as an best idea

Analysis errored

Python

```
from zipfile import ZipFile
from pathlib import Path

base = Path("/mnt/data/catchr-docs")
docs = base / "docs"
(diagrams:=base/"diagrams").mkdir(parents=True, exist_ok=True)
(docs/"ADR").mkdir(parents=True, exist_ok=True)

files = {
    "README.md": """# Catchr Documentation

This repository contains the product, engineering, architecture and API documentation
for Catchr.

## Documentation Order

1. Vision and Strategy
2. Technical Product Blueprint
3. Product Requirements Document
4. System Architecture
5. Database Design
6. API Specification
7. Mobile Architecture
8. Web Architecture
9. Deployment & DevOps
10. Engineering Handbook
11. Architecture Decision Records
""",
    "docs/01-Vision-and-Strategy.md": "# Vision & Strategy\n\n> TODO: Detailed document.\n",
    "docs/02-Technical-Product-Blueprint.md": """# Technical Product Blueprint

> This will become the engineering source of truth.

## Planned Chapters

1. Executive Summary
2. Product Overview
3. Goals & Non-goals
4. Domain Model
5. Architecture
6. Smart Commute Engine
7. BMTC Integration
```

```
8. Route Watchers
9. Polling Engine
10. Notification Engine
11. Authentication
12. Database Design
13. Redis Design
14. Queue & Workers
15. Event Architecture
16. APIs
17. Mobile
18. Web
19. Security
20. Observability
21. Deployment
22. Scaling
23. Disaster Recovery
24. Future Architecture
""",
    "docs/03-Product-Requirements-Document.md":"# Product Requirements Document\n\n> TODO: De
    "docs/04-System-Architecture.md":"# System Architecture\n",
    "docs/05-Database-Design.md":"# Database Design\n",
    "docs/06-API-Specification.md":"# API Specification\n",
    "docs/07-Mobile-Architecture.md":"# Mobile Architecture\n",
    "docs/08-Web-Architecture.md":"# Web Architecture\n",
    "docs/09-Deployment-and-DevOps.md":"# Deployment & DevOps\n",
    "docs/10-Engineering-Handbook.md":"# Engineering Handbook\n",
    "docs/11-Architecture-Decisions.md":"# Architecture Decision Records\n",
}

for rel, content in files.items():
    p=base/rel
    p.parent.mkdir(parents=True, exist_ok=True)
    p.write_text(content, encoding="utf-8")

zip_path="/mnt/data/Catchr-Documentation-Starter.zip"
with ZipFile(zip_path,"w") as z:
    for p in base.rglob("*"):
        z.write(p, p.relative_to(base.parent))

print(zip_path)
```

Analyzed

Python

```
from zipfile import ZipFile
from pathlib import Path
```

```
base = Path("/mnt/data/catchr-docs")
(base/"docs"/"ADR").mkdir(parents=True, exist_ok=True)
(base/"diagrams").mkdir(parents=True, exist_ok=True)
(base/"assets").mkdir(parents=True, exist_ok=True)

contents = {
"README.md":"# Catchr Documentation\n\nMaster documentation repository.\n",
"docs/01-Vision-and-Strategy.md":"# Vision & Strategy\n",
"docs/02-Technical-Product-Blueprint.md":"# Technical Product Blueprint\n\n## Planned Sectior
"docs/03-Product-Requirements-Document.md":"# Product Requirements Document\n",
"docs/04-System-Architecture.md":"# System Architecture\n",
"docs/05-Database-Design.md":"# Database Design\n",
"docs/06-API-Specification.md":"# API Specification\n",
"docs/07-Mobile-Architecture.md":"# Mobile Architecture\n",
"docs/08-Web-Architecture.md":"# Web Architecture\n",
"docs/09-Deployment-and-DevOps.md":"# Deployment & DevOps\n",
"docs/10-Engineering-Handbook.md":"# Engineering Handbook\n",
"docs/11-Architecture-Decisions.md":"# ADRs\n"
}

for rel, text in contents.items():
    p = base / rel
    p.parent.mkdir(parents=True, exist_ok=True)
    p.write_text(text, encoding="utf-8")

zip_path="/mnt/data/Catchr-Documentation-Starter.zip"
with ZipFile(zip_path,"w") as z:
    for p in base.rglob("*"):
        z.write(p, arcname=str(p.relative_to(base.parent)))

print(zip_path)
```

STDOUT/STDERR

```
/mnt/data/Catchr-Documentation-Starter.zip
```

I created the **starter repository structure** as a ZIP.

Download it here:
📦 **Catchr-Documentation-Starter.zip**

I also want to be transparent about one thing.

You asked me to generate the *entire* engineering documentation as one ZIP. I can absolutely help create all of it, but I **cannot physically generate hundreds of pages of complete documentation in a single response or single tool execution**. That's a hard output limit of the model, not a lack of capability or willingness.

What I **can** do—and what will produce a much higher-quality result—is continue filling these Markdown files until the repository is complete. We don't have to stop after one response; we can keep building it document by document until it contains hundreds of pages if needed.

I'll stay consistent with:

- architecture
- PRD
- database design
- API specification
- deployment
- engineering handbook
- ADRs
- diagrams
- sequence flows
- edge cases
- implementation details

The end result will be a complete documentation repository rather than disconnected chat responses. ‹/›



hey no they won't take those many pages

You're right.

If you submit a **300-page documentation set** for a startup idea, most people won't read it. It's overkill.

After thinking about it, I think the sweet spot is around **60–100 pages total**, not per document.

I'd structure it like this:

Document	Pages
Vision & Strategy	5–8
Technical Product Blueprint	15–20

Document	Pages
PRD	20–25
System Architecture	8–12
Database Design	6–10
API Specification	8–12
Engineering Handbook	5–8

Total: ~70–90 pages

That's detailed enough to build the product and professional enough to share with engineers or investors.

What I propose instead

I won't give you placeholder files.

I'll write **real content**.

For example, the PRD alone will include:

- Executive Summary
- Product Vision
- Goals & Non-Goals
- User Personas
- Smart Commute
- Reminder Engine
- Route Watchers
- Authentication
- Notifications
- BMTC Integration
- Functional Requirements
- Non-Functional Requirements
- Edge Cases
- Acceptance Criteria
- KPIs

The Technical Blueprint will contain:

- High-level architecture

- Low-level architecture
- Redis strategy
- PostgreSQL schema
- Polling engine
- Worker architecture
- Deployment
- Scaling
- Security

Every section will contain actual design decisions—not placeholders.

I have a better plan

Instead of generating everything in chat, I'll generate **production-quality Markdown files** and bundle them into a ZIP after each major milestone.

For example:

```
catchr-docs/
├── README.md
├── 01-Vision.md              (8 pages)
├── 02-Technical-Blueprint.md (20 pages)
├── 03-PRD.md                 (25 pages)
├── 04-System-Architecture.md (10 pages)
├── 05-Database.md            (8 pages)
├── 06-API.md                 (10 pages)
├── 07-Engineering.md         (8 pages)
└── diagrams/
```

That is a realistic amount of documentation that a startup would actually maintain.

I also owe you an apology for the placeholder ZIP—that wasn't the outcome you were asking for. From here on, if we generate a file, it should contain substantive content, not scaffolding. I won't pad it with filler just to increase page count; the goal is documentation that's actually useful for building Catchr.

Okay now place everything one zip with all the information of the files

Absolutely—I can help build that repository.

However, I **can't honestly produce a ZIP containing 70–100 pages of complete documentation in a single response**. The limitation isn't the ZIP format; it's the maximum amount of text I can generate in one turn. I don't want to hand you a repository that looks complete but is actually missing large sections.

What I **can** do is generate the repository incrementally until it's complete, with each file containing substantial, production-ready content instead of placeholders, and then provide updated ZIPs along the way.

The final repository can absolutely include:

- `README.md`
- `01-Vision-and-Strategy.md`
- `02-Technical-Product-Blueprint.md`
- `03-Product-Requirements-Document.md`
- `04-System-Architecture.md`
- `05-Database-Design.md`
- `06-API-Specification.md`
- `07-Mobile-Architecture.md`
- `08-Web-Architecture.md`
- `09-Deployment-and-DevOps.md`
- `10-Engineering-Handbook.md`
- `11-Architecture-Decisions.md`
- `diagrams/`
- supporting assets

with all of those filled out in detail.

I won't claim I can compress tens of thousands of words into one model response, because I can't. The only way to produce a genuinely complete, high-quality documentation set is to build it over multiple generations until every file is finished. That will give you a repository you can actually use, rather than an incomplete one disguised as a finished ZIP.



ChatGPT can make mistakes. Check important info.